```python
"""Mavic无人机绕房屋巡逻的Python控制器示例。
   打开机器人窗口可查看摄像头视图。
   此示例演示了如何利用GPS、惯性测量单元和陀螺仪导航到特定世界坐标。
   无人机首先达到指定高度，然后在航点之间巡逻。"""

from controller import Robot
import sys
try:
    import numpy as np
except ImportError:
    sys.exit("警告：未找到'numpy'模块。")


def clamp(value, value_min, value_max):
    """将数值限制在最小值和最大值之间"""
    return min(max(value, value_min), value_max)


class Mavic (Robot):
    # 经验得出的常量值
    K_VERTICAL_THRUST = 68.5  # 此推力可使无人机升空
    K_VERTICAL_OFFSET = 0.6   # 无人机稳定时的垂直偏移量
    K_VERTICAL_P = 3.0        # 垂直PID控制的P常数
    K_ROLL_P = 50.0           # 横滚PID控制的P常数
    K_PITCH_P = 30.0          # 俯仰PID控制的P常数

    MAX_YAW_DISTURBANCE = 0.4  # 最大偏航干扰
    MAX_PITCH_DISTURBANCE = -1 # 最大俯仰干扰
    target_precision = 0.5     # 目标位置与机器人位置之间的精度（米）

    def __init__(self):
        """初始化无人机控制器"""
        Robot.__init__(self)

        self.time_step = int(self.getBasicTimeStep())

        # 获取并启用设备
        self.camera = self.getDevice("camera")
        self.camera.enable(self.time_step)
        self.imu = self.getDevice("inertial unit")
        self.imu.enable(self.time_step)
        self.gps = self.getDevice("gps")
        self.gps.enable(self.time_step)
        self.gyro = self.getDevice("gyro")
        self.gyro.enable(self.time_step)

        # 初始化电机设备
        self.front_left_motor = self.getDevice("front left propeller")
        self.front_right_motor = self.getDevice("front right propeller")
        self.rear_left_motor = self.getDevice("rear left propeller")
        self.rear_right_motor = self.getDevice("rear right propeller")
        self.camera_pitch_motor = self.getDevice("camera pitch")
```

```
        self.camera_pitch_motor.setPosition(0.7)  # 设置摄像头俯仰角度
        motors = [self.front_left_motor, self.front_right_motor,
                  self.rear_left_motor, self.rear_right_motor]
        for motor in motors:
            motor.setPosition(float('inf'))
            motor.setVelocity(1)

        # 初始化状态变量 [X,Y,Z,偏航,俯仰,横滚]
        self.current_pose = 6 * [0]
        self.target_position = [0, 0, 0]  # 目标位置
        self.target_index = 0             # 当前目标航点索引
        self.target_altitude = 0          # 目标高度

    def set_position(self, pos):
        """
        设置机器人的新绝对位置
        参数:
            pos (list): [X,Y,Z,偏航,俯仰,横滚]当前绝对位置和角度
        """
        self.current_pose = pos

    def move_to_target(self, waypoints, verbose_movement=False,
verbose_target=False):
        """
        将机器人移动到给定坐标
        参数:
            waypoints (list): X,Y坐标列表
            verbose_movement (bool): 是否打印剩余角度和距离
            verbose_target (bool): 是否打印目标信息
        返回:
            yaw_disturbance (float): 偏航干扰（负值表示向右）
            pitch_disturbance (float): 俯仰干扰（负值表示向前）
        """

        if self.target_position[0:2] == [0, 0]:  # 初始化
            self.target_position[0:2] = waypoints[0]
            if verbose_target:
                print("第一个目标: ", self.target_position[0:2])

        # 检查是否到达目标位置（在允许误差范围内）
        if all([abs(x1 - x2) < self.target_precision for (x1, x2) in
zip(self.target_position, self.current_pose[0:2])]):
            self.target_index += 1
            if self.target_index > len(waypoints) - 1:
                self.target_index = 0
            self.target_position[0:2] = waypoints[self.target_index]
            if verbose_target:
                print("到达目标! 新目标: ", self.target_position[0:2])

        # 计算目标方向角度（-π到π）
        self.target_position[2] = np.arctan2(
            self.target_position[1] - self.current_pose[1],
            self.target_position[0] - self.current_pose[0])
        # 计算剩余转向角度（-2π到2π）
```

```
        angle_left = self.target_position[2] - self.current_pose[5]
        # 标准化转向角度到（-π到π]
        angle_left = (angle_left + 2 * np.pi) % (2 * np.pi)
        if (angle_left > np.pi):
            angle_left -= 2 * np.pi

        # 根据剩余角度计算转向干扰
        yaw_disturbance = self.MAX_YAW_DISTURBANCE * angle_left / (2 *
np.pi)
        # 非比例递减函数计算俯仰干扰
        pitch_disturbance = clamp(
            np.log10(abs(angle_left)), self.MAX_PITCH_DISTURBANCE, 0.1)

        if verbose_movement:
            distance_left = np.sqrt(((self.target_position[0] -
self.current_pose[0]) ** 2) + (
                (self.target_position[1] - self.current_pose[1]) ** 2))
            print("剩余角度: {:.4f}, 剩余距离: {:.4f}".format(
                angle_left, distance_left))
        return yaw_disturbance, pitch_disturbance

    def run(self):
        """主控制循环"""
        t1 = self.getTime()

        roll_disturbance = 0  # 横滚干扰
        pitch_disturbance = 0 # 俯仰干扰
        yaw_disturbance = 0   # 偏航干扰

        # 指定巡逻坐标点
        waypoints = [[-30, 20], [-60, 20], [-60, 10], [-30, 5]]
        self.target_altitude = 15  # 目标高度（米）

        while self.step(self.time_step) != -1:
            # 读取传感器数据
            roll, pitch, yaw = self.imu.getRollPitchYaw()
            x_pos, y_pos, altitude = self.gps.getValues()
            roll_acceleration, pitch_acceleration, _ =
self.gyro.getValues()
            self.set_position([x_pos, y_pos, altitude, roll, pitch, yaw])

            if altitude > self.target_altitude - 1:
                # 达到目标高度后，计算前往航点的干扰量
                if self.getTime() - t1 > 0.1:
                    yaw_disturbance, pitch_disturbance =
self.move_to_target(
                        waypoints)
                    t1 = self.getTime()

            # 计算各方向控制输入
            roll_input = self.K_ROLL_P * clamp(roll, -1, 1) +
roll_acceleration + roll_disturbance
            pitch_input = self.K_PITCH_P * clamp(pitch, -1, 1) +
pitch_acceleration + pitch_disturbance
```

```
            yaw_input = yaw_disturbance
            clamped_difference_altitude = clamp(self.target_altitude -
altitude + self.K_VERTICAL_OFFSET, -1, 1)
            vertical_input = self.K_VERTICAL_P *
pow(clamped_difference_altitude, 3.0)

            # 计算各电机输入
            front_left_motor_input = self.K_VERTICAL_THRUST +
vertical_input - yaw_input + pitch_input - roll_input
            front_right_motor_input = self.K_VERTICAL_THRUST +
vertical_input + yaw_input + pitch_input + roll_input
            rear_left_motor_input = self.K_VERTICAL_THRUST +
vertical_input + yaw_input - pitch_input - roll_input
            rear_right_motor_input = self.K_VERTICAL_THRUST +
vertical_input - yaw_input - pitch_input + roll_input

            # 设置电机速度
            self.front_left_motor.setVelocity(front_left_motor_input)
            self.front_right_motor.setVelocity(-front_right_motor_input)
            self.rear_left_motor.setVelocity(-rear_left_motor_input)
            self.rear_right_motor.setVelocity(rear_right_motor_input)


# 使用此控制器时，basicTimeStep应设置为8，defaultDamping
# 的线性和角度阻尼都应设置为0.5
robot = Mavic()
robot.run()
```